

Instituto Superior Politécnico de Tecnologias e Ciências
(ISPTEC)

Departamento de Engenharia e Tecnologias

Engenharia de Software II

Licenciatura em Engenharia Informática

2025/2026

Lab #04 — Documento Técnico

Sistema de Gestão Financeira Pessoal

SmartBudget

Jofre Jaime Jamuinda Mutumbungo

N.º 20230394

Grupo 3

Luanda, Maio de 2026

1. Descrição do Sistema

O SmartBudget é uma aplicação web de gestão financeira pessoal desenvolvida no âmbito do Laboratório #04 da disciplina de Engenharia de Software II. O sistema tem como objectivo principal permitir que um utilizador registe, organize e acompanhe as suas receitas e despesas de forma simples e clara, tendo acesso a um resumo financeiro actualizado em tempo real.

A aplicação é composta por dois componentes principais: um **frontend** desenvolvido em Angular 21 com componentes independentes (standalone), e um **backend** construído em PHP 8.2 puro, sem recurso a frameworks, expondo uma REST API com respostas em formato JSON. A persistência de dados é assegurada por uma base de dados relacional MySQL 8.0.

1.1 Stack Tecnológica

Camada	Tecnologia
Frontend	Angular 21 — Standalone Components, TypeScript, SCSS
Backend	PHP 8.2 puro, strict types, arquitectura em camadas
Base de dados	MySQL 8.0 com InnoDB e índices otimizados
Autenticação	JWT (HS256) com validade de 24 horas
Internacionalização	ngx-translate — Português e Inglês
Ícones	Tabler Icons (webfont, outline only)

2. Funcionalidades Implementadas

Todas as funcionalidades obrigatórias definidas no enunciado foram implementadas. O sistema cumpre integralmente os requisitos técnicos estabelecidos.

2.1 Autenticação e Gestão de Conta

- Registo de utilizador com validação de email e hash de password (bcrypt, custo 12)
- Login com geração de token JWT (24h de validade, assinatura HS256)
- Logout com limpeza de token no cliente
- Edição de perfil: nome e endereço de email
- Alteração de password com verificação da senha actual
- Eliminação de conta com confirmação por password

2.2 Gestão de Transacções

- Criação de receitas e despesas com: valor, tipo, categoria, descrição, data e notas
- Listagem paginada com suporte a filtros por tipo, categoria e intervalo de datas
- Edição e eliminação de transacções existentes
- Exportação de transacções para ficheiro CSV com codificação UTF-8

2.3 Gestão de Categorias

- Criação de categorias privadas por utilizador (nome, ícone, tipo, descrição)
- Edição e arquivo de categorias (soft delete via campo is_active)
- Filtragem de categorias por tipo (receita / despesa)

2.4 Dashboard Financeiro

- Resumo geral: saldo total acumulado, total de receitas e total de despesas
- Resumo mensal: receitas, despesas e saldo líquido do mês corrente
- Últimas transacções registadas
- Agregação por categoria para o mês actual

2.5 Requisitos Transversais

- Interface responsiva (mobile, tablet e desktop)
- Tema claro e escuro com alternância manual e detecção automática via `prefers-color-scheme`
- Suporte a dois idiomas: Português e Inglês (`ngx-translate`)
- Exportação de relatório em CSV a partir do Dashboard

3. Decisões Técnicas

3.1 Backend sem Framework

A decisão de desenvolver o backend em PHP puro, sem recurso a frameworks, foi motivada pelos requisitos do enunciado e serviu como exercício de compreensão da arquitectura em camadas. O sistema implementa os padrões *Repository* e *Controller-Service-Repository*, com injeção de dependências manual e um router baseado em expressões regulares. Esta abordagem torna a separação de responsabilidades explícita e auditável.

3.2 Segurança

Toda a comunicação autenticada é protegida por JWT com assinatura HS256. As passwords são armazenadas com `bcrypt` (factor de custo 12). Todas as queries à base de dados utilizam *prepared statements* via PDO, eliminando vulnerabilidades de injeção SQL. Os inputs são validados e sanitizados antes de qualquer processamento.

3.3 Base de Dados

O schema MySQL é composto por três tabelas principais — *users*, *transactions* e *categories* — com índices sobre os campos de pesquisa mais frequentes (*user_id*, *category_id*, *date*, *email*). A integridade referencial é garantida por chaves estrangeiras com *ON DELETE CASCADE*. O motor InnoDB assegura suporte a transacções ACID.

3.4 Frontend — Angular 21

O frontend foi estruturado com componentes standalone, eliminando a necessidade de NgModules e tornando a árvore de dependências mais clara. A comunicação com a API é centralizada em serviços Angular com *HttpClient*. Um *JWT Interceptor* injeta automaticamente o token de autenticação em todos os pedidos protegidos. O *AuthGuard* protege as rotas que requerem autenticação.

3.5 Design System

Foi definido um sistema de design próprio baseado em tokens CSS (variáveis), com paleta de cores semântica (verde primário para saldo positivo, vermelho para despesas), escala de espaçamentos em múltiplos de 4px e border radius consistentes. As fontes escolhidas foram *Sora* (headings e valores numéricos) e *Inter* (corpo de texto e interface), combinação que confere um registo profissional e legível. O suporte a modo escuro é implementado através do selector `[data-theme="dark"]` aplicado no `documentElement` pelo *ThemeService*.

4. Dificuldades Encontradas

4.1 CORS e Comunicação Frontend–Backend

A configuração inicial dos cabeçalhos CORS no PHP gerou erros nos pedidos *pre-flight* do Angular. A solução implicou adicionar os cabeçalhos adequados na resposta a pedidos OPTIONS antes do encaminhamento pelo router, e garantir que os cabeçalhos *Access-Control-Allow-** fossem enviados em todas as respostas, incluindo as de erro.

4.2 Gestão do Estado de Autenticação no Angular

Garantir que o estado de autenticação fosse consistente entre componentes e resistente a actualizações de página exigiu a implementação de um *AuthService* com um *BehaviorSubject*. O *AuthGuard* foi adaptado para verificar a validade do token e redirecionar para o login em caso de expiração, sem que o utilizador perdesse o contexto de navegação.

4.3 Exportação CSV com Codificação

A exportação de ficheiros CSV com caracteres especiais (letras acentuadas) apresentou problemas de codificação ao abrir os ficheiros no Microsoft Excel. A solução adoptada foi adicionar o BOM UTF-8 (`\xEF\xBB\xBF`) no início do conteúdo do ficheiro e definir o cabeçalho HTTP *Content-Type: text/csv; charset=utf-8* na resposta da API.

4.4 Reactive Forms e Validação

A integração dos formulários Angular com o padrão *ReactiveFormsModule* exigiu uma reestruturação dos componentes de formulário inicialmente construídos com *template-driven forms*. Apesar do esforço adicional, o resultado melhorou significativamente a testabilidade e a gestão dos estados de validação, em particular a exibição contextual de mensagens de erro.

5. Conclusão

O SmartBudget cumpre todos os requisitos técnicos obrigatórios definidos no enunciado do laboratório: autenticação completa, base de dados relacional com mais de três tabelas, interface responsiva, suporte a dois idiomas, modo claro e escuro, e exportação de dados em CSV.

O projecto foi desenvolvido com foco na organização do código, separação de responsabilidades e qualidade técnica. A experiência de construir um backend em PHP puro e um frontend Angular comunicando através de uma API REST representou um exercício completo do ciclo de desenvolvimento de um sistema web distribuído, desde a modelação da base de dados até à interface de utilizador.

As dificuldades encontradas contribuíram para uma compreensão mais profunda de aspectos como segurança HTTP, gestão de estado em aplicações SPA e boas práticas de codificação em PHP e Angular.